

Reproducing AWQ for Efficient 4-bit LLM Inference on Qwen2.5-3B-Instruct

James Condos, 1080595 and Sithira Jayasinghe, 884040

ABSTRACT

Large language models achieve strong performance across natural language tasks, but their high memory requirements and inference costs make them difficult to deploy on limited hardware. This project investigates Activation-Aware Weight quantisation (AWQ) as an efficient compression method for reducing the memory footprint of large language models while preserving model quality as Lin et al set out in their paper [10]. AWQ is motivated by the observation that not all model weights contribute equally to performance, as a small subset of activation-sensitive channels have a disproportionately large effect on output accuracy. By identifying and protecting these important channels during low-bit quantisation, AWQ aims to achieve better accuracy–efficiency. In this project, we reproduce and evaluate an AWQ-style 4-bit quantisation workflow using **Qwen2.5-3B-Instruct** as the target model. The quantised model is compared against a baseline full-precision model and a simpler low-bit quantisation approach. Performance is evaluated using multiple practical metrics, including GPU memory usage, inference latency, tokens per second, perplexity, and task accuracy on selected benchmark datasets. Additional experiments examine how calibration set size and quantisation configuration affect model quality and efficiency. We will first however introduce the basics of LLM’s and their packages in parts 1 to 5 which makes use of the model **Mistral-7B-Instruct-v0.3**.

1. PART A - LLM PIPELINE

1.1. Autoregressive Text Generation

When running `pipeline("text-generation")` without specifying a model, warning messages were shown as expected. It should be noted that these warnings were not fatal errors. The main warning was that no model was supplied, so Hugging Face defaulted to a GPT-2 text-generation model, and this also did not stop the cell from running. To remove this warning, the following command was run, this command explicitly specified the model and tokenizer:

- ```
generator = pipeline("text-generation",
model="gpt2", tokenizer="gpt2")
```

Another warning was related to `pad_token_id` being set to `eos_token_id`. GPT-2 does not have a separate padding token because it was trained as an autoregressive language model. It instead predicts the next token from the previous tokens rather than processing batches in the same way as encoder models. The `eos_token_id` is the end-of-sequence token, which for GPT-2 is commonly token ID 50256. For open-ended generation, Hugging Face often uses this as the padding token to avoid errors. These errors were eliminated by instead running the following, which then gave the respective output:

- ```
generator(
    "In this LLM project, we will teach you how to",
    max_new_tokens = 50,
    pad_token_id=generator.tokenizer.eos_token_id)
```

GPT-2 Generated Output

In this LLM project, we will teach you how to apply the LLM programming language from the programming language-oriented programming paradigm. We will use C++, C# and Visual Basic as our languages. We will use a variety of algorithms and libraries to bring these two to life.' example.

The output is easy to read, but the quality is not very good. It reads as proper English initially, but the meaning is quite confused. The model says “LLM programming language”, which

does not really make sense because in our project, LLM means Large Language Model, not a programming language. It also talks about C++, C# and Visual Basic, which are not related to our Hugging Face LLM project. So the text is grammatically okay, but it does not answer our prompt correctly. This shows that GPT-2 can generate text that sounds fluent, but it does not always understand the actual context and can make up irrelevant information.

1.2. Sentiment Analysis

BERT stands for Bidirectional Encoder Representations from Transformers, which is used for classification such as sentiment analysis and question answering. We will note however that BERT does not answer questions in the general sense of generation. BERT answers questions through token classification, looking at the important tokens in a context of text, and makes a prediction. This is important to note, as BERT is used as an encoder, compressing text all at once, and making a prediction or classification based on the context.

Distilbert is a much smaller and faster version of BERT, using less memory, and making use of **knowledge distillation**, a compression technique, where distilbert is trained to reproduce the behaviour and performance of a larger BERT model.

`Distilbert-base-uncased-finetuned-sst-2-english` is the specific model that Hugging Face uses by default for the sentiment-analysis pipeline.

We recall that the output of our sentiment pipeline using BERT looks like the following:

- ```
[{'label': 'POSITIVE', 'score': 0.8745}]
```

The score is the model’s confidence in a certain prediction. It is a probabilistic value between 0 and 1. A score close to 1 means the model is very confident. A score closer to 0.5 means the model is less certain.

We can see that the sentiment analysis works well in the obvious cases for sentences 1 and 2 below in Table 1, however, sentence 3 could be interpreted as more Negative. This is because BERT is a binary classifier and does not have a neutral label. This shows a limitation of sentiment models, as they may struggle with mixed or neutral opinions which can be sensitive to wording.

**Table 1.** Sentiment Analysis

| Sentence                                                                           | Label    | Score  |
|------------------------------------------------------------------------------------|----------|--------|
| "I really enjoyed this subject."                                                   | POSITIVE | 0.9999 |
| "This project is confusing and frustrating."                                       | NEGATIVE | 0.9996 |
| "The lecture had many issues which were disliked, very slow, but still just okay." | POSITIVE | 0.9986 |

### 1.3. Exploring Other Hugging Face Pipelines

The first pipeline was a named entity recognition pipeline. This pipeline looks through a sentence and identifies important named things, such as organisations, places, and people. We see below that the **PER** and the second **ORG** were found correctly, however, the first **ORG** was identified as *E*, which was incorrect. This may be due to the fact that the model is not aware of the Universities subject code formats, leading to errors and non-confident scores.

#### Named Entity Pipeline

**Given Text:** My names is James Condos, and I take ELEN90088 at the University of Melbourne."

- **Entity:** James Condos, Type: PER, Score: 0.9987
- **Entity:** E, Type: ORG, Score: 0.4335
- **Entity:** University of Melbourne, Type: ORG Score: 0.9969

The second pipeline explored was a question-answering pipeline. This pipeline takes two inputs. A question and a paragraph of context. This pipeline searches inside the given paragraph and tries to extract the part of the text that best answers the question. Our example is shown below:

#### Question-Answering Pipeline

**Given Text:** "System Optimisation and Machine Learning is a subject that first introduces convex optimisation problems, how to solve them using gradient descent and other algorithms. It then introduces machine learning concepts and techniques, such as supervised learning, unsupervised learning, and deep learning."

- **Question:** What is taught in the System Optimisation and Machine Learning subject?
- **Answer:** convex optimisation problems
- **Score:** 0.1854

It can be observed that this result is partially correct because convex optimisation problems are mentioned in the given text, however, it is incomplete because the subject also covers gradient descent, other algorithms, and machine learning topics such as supervised, unsupervised, and deep learning, justified by the low score of 0.1854.

The first call to `pipeline()` downloads cached Hugging Face files, loads the tokeniser and model weights, allocated memory, places the model on our machine, and sets up task-specific pre-processing/postprocessing. Later calls are faster because these files and objects are cached or already loaded, so the workflow mostly consists of tokenisation, model inference, and output decoding/postprocessing.

## 2. PART B - LLM WORKFLOW

### 2.1. Tokeniser

After exploring different tokenisation, we observed that words such as **Advancements** was split up into "Adv", "ance" and "ments", **Knowledgeable** was split up into "Know", "ledge" and "able", with both also including the BOS (Beginning-Of-Sequence) token. We recognise that tokenisation is a good approach for preprocessing, because representing every full word with a unique integer would create a huge vocabulary and struggle with rare or unseen words. This is because most words share common bases, prefixes and suffixes to others.

### 2.2. Model

We can check whether our loaded model (in this case Mistral-7B-Instruct-v0.3) via the python snippet `next(model_raw.parameters()).device`, which displays "cuda" for being loaded onto our Nvidia GPU.

Exploring the model, we see that it employs a transformer-based architecture, which consists of 32 stacked transformer layers, each with a hidden dimension of 4,096 [8]. This uses 32 attention heads, which allows the model to focus on different relationships between tokens. Conversely, the model does not employ a sliding window mechanism, meaning the prediction of the next token makes use of all tokens beforehand [9]. This information was found using code snippet (1).

```
Check the device of the model parameters
print(next(model_raw.parameters()).device)

config = model_raw.config

print("Architecture:", config.architectures)
print("Model type:", config.model_type)
print("Number of layers:", config.num_hidden_layers)
print("Hidden size:", config.hidden_size)
print("Attention heads:", config.num_attention_heads)
print("Key/value heads:", config.num_key_value_heads)
print("Sliding window:", config.sliding_window)
print("Vocabulary size:", config.vocab_size)
```

**Code 1.** MistralForCausalLM Architecture Information

## 3. PART C - INFERENCE ENGINEERING

### 3.1. Text Response with LLM

To make sure the output talks about Large Language Models, we would avoid relying only on the abbreviation "LLM". We would write out "Large Language Models" in the prompt and add context such as "AI language models", "tokens", "transformers", and "text generation". This reduces ambiguity and steers the model toward the intended topic.

We also varied `temperature` between 0.1 and 1, `max_new_tokens` and `do_sample` as shown in appendix A for a more thorough understanding.

The three outputs show clear differences in tone, structure, and randomness. With `temperature = 0.1`, the response is formal and highly structured. It uses a predictable numbered outline and stays closely focused on standard LLM topics, but feels slightly generic.

With `temperature = 1`, the response is more conversational and creative. It uses phrases like “Hey there, students!” and includes more varied examples, such as RNNs, CNNs, Pandas, NumPy, and softmax. However, it is slightly less precise as it gives broader, less focused explanations.

With `do_sample_state = False`, the response is deterministic. It follows a very logical sequence from machine learning basics to deep learning, transformers, applications, ethics, and projects, and is both consistent and comprehensive.

3.2. Sentiment Analysis with LLM

The results showed that the model did respect the instruction to output only one word in the zero-shot setting. The raw zero-shot outputs were all either Positive or Negative, so the model followed the requested output format without post-processing. This is likely because the prompt was very direct with use of `do_sample = False`.

Zero-Shot Vs Few-Shot LLM Sentiment

Review: “I was furious when the concert ended because I wanted another hour.”

• Zero-Shot: Negative

• Zero-Shot Tokens: 47

• Few-Shot: Positive

• Few-Shot Tokens: 244

Zero-shot labelled this as Negative, while few-shot labelled it as Positive. The zero-shot model likely focused too strongly on the negative word “furious”. The few-shot examples taught the model that negative emotion words can still express positive sentiment when the reason is that the experience was enjoyable and the reviewer wanted more of it.

On average in this experiment, zero-shot used about 50.7 tokens, while few-shot used about 247.8 tokens, so the few-shot prompt was roughly 4.9x longer.

4. PART D - EFFICIENT LLM COMPRESSION

4.1. Implementation: Bitsandbytes Library

Although the model weights are stored in 4-bit precision, the actual computations are not usually performed directly using 4-bits. The 4-bit weights are used to reduce memory storage, and during inference, these compressed weights are dequantised into higher precision, and in our case, `torch.bfloat16`, for matrix multiplications.

For our project, the 16-bit computation is useful because 4-bit values are too low precision for stable numerical operations. If all calculations were done in 4-bit, rounding errors would accumulate quickly and the model output quality would degrade. Using `bfloat16` uses much less memory than `float32`, but still has enough numerical range and precision to keep inference stable.

4.2. VRAM Vs Disk Size

Loading the Mistral-7B-Instruct-v0.3 model would use approximately:

- 7 billion parameters x 4 bits = 28 billion bits = 3.5 GB

However, our measured VRAM usage was about 5.9 GB, which is higher than the theoretical minimum. This is because the 3.5

GB estimate only counts the quantised model weights. During inference, extra VRAM is needed for quantisation metadata, CUDA/PyTorch overhead, temporary computation buffers and intermediate computations in the neural network (activations).

As such, the 4-bit model still provides a large memory reduction compared with the 16-bit baseline, but the actual VRAM usage is higher than 3.5 GB because running the model requires more memory than just storing the compressed parameters.

4.3. Perplexity

Perplexity was used to compare the predictive quality of the original 16-bit Mistral model and the compressed 4-bit NF4 model. The input text was first tokenized, then passed through the language model with the same token IDs used as labels. This allows the model to compute the average next-token prediction loss. Perplexity was then calculated as:

$$PPL = e^{(loss)}$$
 (1)

where lower perplexity indicates that the model assigns higher probability to the text and is less “surprised” by it.

Table 2. Perplexity comparison between the 16-bit and 4-bit NF4 Mistral models.

| Model             | Perplexity |
|-------------------|------------|
| 16-bit Mistral    | 2.4402     |
| 4-bit NF4 Mistral | 2.3810     |

For the selected Wikipedia-style sample, the 16-bit model achieved a perplexity of 2.4402, while the 4-bit NF4 model achieved a perplexity of 2.3810. These values are very close, indicating that quantisation did not noticeably degrade the model’s ability to predict the sample text. Interestingly, the 4-bit model gave a slightly lower perplexity on this short example, but this should not be interpreted as the quantised model being generally better. Since the test text is short, small numerical differences can occur due to rounding, quantisation effects, and the limited sample size.

Overall, the perplexity results suggest that 4-bit NF4 quantisation preserved the language modelling performance of the original model very well for this example, while greatly reducing storage size. A more reliable evaluation would use a larger dataset and the sliding-window perplexity method recommended by Hugging Face.

4.4. Text Generation Quality

To evaluate whether quantisation affected text-generation quality, we first compared the 16-bit baseline model with the 4-bit NF4 quantised model on a sarcasm explanation task. Both models were given the same prompt as shown below.

Prompt and Question

Prompt: "Explain why this review is sarcastic in 3–5 sentences. Focus on the contrast between the literal wording and the intended meaning"

Quote: Review: “If you enjoy watching paint dry, you’ll

love this movie!"

- **16-bit Explanation:** This review is sarcastic because, contrary to the literal wording, the reviewer is implying that the movie being reviewed is incredibly dull and boring, similar to the mundane activity of watching paint dry. The intended meaning is that the movie is not enjoyable or entertaining.
- **4-bit NF4 Explanation:** This review is sarcastic because the statement "If you enjoy watching paint dry, you'll love this movie!" is intended to be ironic. The reviewer is suggesting that the movie is so dull, slow, or uninteresting that it is comparable to the mundane and tedious activity of watching paint dry. The use of the word "enjoy" in this context is sarcastic, as no one typically finds watching paint dry enjoyable.

In terms of coherence, both the 16-bit and 4-bit models were able to identify the main sarcastic meaning of the sentence. The 16-bit model produced a slightly more polished and fluent explanation, while the 4-bit NF4 model was still able to preserve the key interpretation.

Overall, the comparison suggests that 4-bit NF4 quantisation does not significantly damage the model's practical generation quality for this task. Although the 16-bit model may produce slightly smoother language, the 4-bit model still captured the intended sarcastic meaning.

## 5. PART E - EFFICIENT LLM PARAMETER FINE TUNING (PEFT)

### 5.1. Pre-training vs Fine-tuning

For this exploration into PEFT, we instead make use of the `Phi-3.5-mini-instruct` model.

Large language models usually use two main training stages, which are given by *pre-training* and *fine-tuning*. In pre-training, the model is trained on a very large amount of text so that it can learn general language patterns, grammar, facts and relationships between words.

Fine-tuning happens after pre-training. Instead of training the model from the beginning again, the already pre-trained model is trained on a smaller and more specific dataset. This helps the model become better at a particular task, such as answering questions, following instructions, summarising text or writing in a certain style [12]. For example, instruction fine-tuning can make a general language model more useful because it teaches the model how to respond to user prompts more clearly.

Overall, pre-training gives the model its general ability, while fine-tuning makes it more useful for a specific application. This means fine-tuning is important because a model may have strong general language ability but still need extra training to follow instructions or perform well on a particular task.

On a more interesting note, Sun and Dredze explored this relationship in more depth, and found that "the benefits of fine-tuning an LLM could exceed the benefits of continued pretraining" [11], emphasising its importance.

### 5.2. Standard LoRA (16-bit)

Hu et al highlights an important method for PEFT in [3]. The LoRA rank  $r$  controls the capacity of the low-rank adapter. In

full fine-tuning, a model learns a full update matrix  $\Delta W$ , so that  $W_{\text{new}} = W + \Delta W$ . LoRA instead approximates this update using two smaller matrices,  $\Delta W = BA$ , where  $A \in \mathbb{R}^{r \times d_{\text{in}}}$  and  $B \in \mathbb{R}^{d_{\text{out}} \times r}$ . Thus, the effective weight becomes  $W_{\text{new}} = W + BA$ . Increasing  $r$  increases the number of trainable parameters and the expressiveness of the adapter, while decreasing  $r$  reduces memory and computation cost. This is shown more explicitly in Table 3. Another tunable parameter `lora_alpha`,  $\alpha$ , provides a scale factor for  $BA$ , in the form  $\frac{\alpha}{r}$ , however, this has no real effect on the number of trainable parameters.

**Table 3.** Trainable Parameters with different values for  $r$  using LoRA

| Hyper Parameter Value | Trainable Percentage |
|-----------------------|----------------------|
| $r = 60$              | 0.61365%             |
| $r = 30$              | 0.30777%             |
| $r = 16$              | 0.16438%             |
| $r = 5$               | 0.051464%            |

### 5.3. QLoRA Training Loop

No hyperparameter tuning was performed for this experiment. We used a fixed QLoRA configuration with rank  $r = 16$ , `lora_alpha` = 32, a learning rate of  $2 \times 10^{-4}$ , and `max_steps` = 100, as to demonstrate the use and computational properties of the method.

After applying QLoRA to the Phi-3.5 model, the number of trainable parameters were equivalent to what was shown in Table 3, with 6,291,456 trainable parameters and 3,827,371,008 total parameters.

Compared with a 3.5 billion parameter model, the number of trained parameters is approximately only 0.1644%, and as such, QLoRA is highly parameter-efficient.

#### 5.3.1. VRAM Usage and Adapter Size

During the `trainer.train()` call, the observed GPU memory usage was approximately:

3438 MiB

on an 8 GB laptop GPU. This is well below the available GPU memory:

$$3438 \text{ MiB} < 8151 \text{ MiB}$$

Therefore, this QLoRA training run can be performed on a standard laptop GPU with approximately 8 GB of VRAM. The base model is loaded in 4-bit precision, while only the LoRA adapter parameters are trained.

We now look at our adapter size. We note that our saved adapter weight file was given by:

$$\text{adapter\_model.safetensors} = 24.008 \text{ MB}$$

Our cached Phi-3.5 baseline was approximately given by:

$$7.29 \text{ GB}$$

Thus, the adapter weight file was only:

$$\frac{24.008}{7288.16} \times 100 \approx 0.329\%$$

of the full Phi-3.5 size, demonstrating that QLoRA is storage-efficient, and that only the small adapter weights to be saved

again.

### 5.3.2. Perplexity and Text Generation Quality

The QLoRA model achieved a perplexity of 12.5732. This suggests that the fine-tuned model assigns reasonably confident probability to the evaluation text, recalling that the fine-tuned model only used 100 steps.

## 6. PART F - AWQ FOR EFFICIENT 4-BIT LLM INFERENCE: MINI PROJECT

### 6.1. Mini-Project Motivation

This final section extends beyond the notebook tasks by reproducing and evaluating a research method for efficient LLM inference by Activation-aware Weight quantisation (AWQ), proposed by Lin et al. [10].

Qwen2.5-3B-Instruct was selected as the target model because it provides a clean and controlled setting for evaluating AWQ. The Qwen2.5 model family includes both a full-precision instruction-tuned model and an official AWQ-quantised counterpart, allowing the comparison to focus on the effects of quantisation. This makes the evaluation more consistent.

### 6.2. Research Questions

This mini-project investigates the following research questions:

#### Research Questions

1. How much GPU memory does AWQ reduce compared with full-precision inference?
2. Does AWQ preserve model quality better than simple 4-bit quantisation and reduce error compared to basic 4-bit quantisation?
3. Does AWQ improve inference latency or tokens per second under the available hardware constraints?
4. How sensitive is AWQ performance to the calibration samples used during quantisation?
5. Can activation-coverage-based calibration selection improve AWQ performance compared with random calibration?

These questions evaluate both sides of the compression trade-off. A useful quantisation method should reduce memory and computational cost, but should not significantly damage the model's ability to predict or generate useful text.

**NOTE: Although we are explicitly restating the method Lin et al. uses in their paper, this is present to ensure that we understand the quantisation method presented.**

### 6.3. AWQ Method Overview

As explored in Section 4 quantisation reduces the numerical precision used to represent model weights. For a linear layer with weight matrix  $W$  and input activation  $x$ , the full-precision output is:

$$y = Wx. \quad (2)$$

Lin et al. defines their quantisation function to be a simple well defined function being given by **Round-To-Nearest**:

$$Q(w) = \Delta \cdot \text{Round}\left(\frac{w}{\Delta}\right) \quad (3)$$

$$\Delta = \frac{\max |w|}{2^{N-1}} \quad (4)$$

Where  $N$  is our number of quantisation bits.

After quantisation, the original weight matrix  $W$  is replaced by our quantised weights given by:

$$\hat{W} = Q(W) \quad (5)$$

And thus gives us the following output of a singular layer:

$$\hat{y} = \hat{W}x \quad (6)$$

And error by:

$$\text{Error} = y - \hat{y} = (W - \hat{W})x \quad (7)$$

#### Significance of Methodology

This expression shows why activations matter. A quantisation error in a channel with large activation magnitude can have a larger effect on the final output than the same quantisation error in a channel with small activation magnitude.

Therefore, minimising weight error alone may not be sufficient. AWQ uses calibration activations to identify channels that are more important to the model's outputs and protects these channels during low-bit quantisation.

As such, Lin et al. proposed AWQ method applies the usual low-bit quantisation function to a scaled version of the weight, written as  $Q(ws)$ , rather than directly quantising  $w$ . The scale factor  $s$  is chosen per channel, with larger values assigned to salient channels that are more important to the output. This is outlined in more detail below:

$$Q(w \cdot s) \cdot \frac{x}{s} = \Delta' \cdot \text{Round}\left(\frac{ws}{\Delta'}\right) \cdot \frac{x}{s} \quad (8)$$

$$\Delta = \frac{\max |ws|}{2^{N-1}} \quad (9)$$

Where  $w \in \mathbf{w}$ .

Lin et al. defines a "**salient channel**" as a channel in which its quantisation error is expected to have a "disproportionately large effect on the layer output".

We should note that the AWQ paper argues that the expected rounding error is approximately unchanged by scaling, so the error in rounding between  $ws$  and  $\frac{ws}{\Delta'}$  is expected to be quite small. Lin et al. also assumes that the *quantisation step*,  $\Delta$  to be approximated by  $\Delta'$ , and that  $s > 1$ .

The error in quantisation methods is presented below:

$$\text{RTN Error}(Q(w)x) = \Delta \cdot \left(\text{Round}\left(\frac{w}{\Delta}\right) - \frac{w}{\Delta}\right) \cdot x \quad (10)$$

$$\text{AWQ Error}(Q(w \cdot s)\left(\frac{x}{s}\right)) = \Delta' \cdot \left(\text{Round}\left(\frac{ws}{\Delta'}\right) - \frac{ws}{\Delta'}\right) \cdot \frac{x}{s} \quad (11)$$

And as such, the ratio between these errors is approximately:

$$\frac{\text{AWQ Error}}{\text{RTN Error}} \approx \frac{1}{s} \quad (12)$$

As this ratio is less than one, this means that scaling a salient weight channel can reduce its expected output quantisation error by  $\frac{1}{s}$ .

More formally, Lin et al. defines the following loss function and optimisation problem in matrix form adapted from equation

8 given by:

$$\mathcal{L}(\mathbf{s}) = ||Q(\mathbf{W} \cdot \text{diag}(\mathbf{s}))(\text{diag}(\mathbf{s})^{-1} \cdot \mathbf{X}) - \mathbf{W}\mathbf{X}|| \quad (13)$$

$$\mathbf{s}_{\text{optimal}} = \arg \min_{\mathbf{s}} \mathcal{L}(\mathbf{s}) \quad (14)$$

However, we should note that this is not a simple optimisation problem. Our quantisation function used in our experiments and results has inherent discontinuities from jumping between rounded values, meaning that it is **not a smooth function**. This implies that our loss function itself is not differentiable.

### 6.3.1. Choosing the Scaling Factors

Lin et al. proposes that AWQ use a simpler search space based on activation statistics from the above implication. Let  $s_x$  denote the average activation magnitude for each input channel, computed from a small calibration set. AWQ then defines the scaling vector as:

$$\mathbf{s} = \mathbf{s}_X^\alpha, \quad (15)$$

where  $\alpha$  is a scalar search parameter. The best value of  $\alpha$  is selected by:

$$\alpha_{\text{optimal}} = \arg \min_{\alpha} L(\mathbf{s}_X^\alpha). \quad (16)$$

When  $\alpha = 0$ , all channels have scale 1, so no scaling is applied. When  $\alpha = 1$ , the scaling is directly proportional to the average activation magnitude, giving the most aggressive scaling in our search space.

However, the scale cannot be made arbitrarily large. If  $s$  is too large, the maximum value in our quantisation can increase, causing  $\Delta'$  to increase. A larger  $\Delta'$  makes the quantisation coarser and can increase the error for non-salient channels.

AWQ is hypothesised to improve low-bit quantisation because the damage caused by weight quantisation depends not only on the weight error, but also on the activation that multiplies that weight. A channel with large activation magnitude can amplify quantisation error and cause a large change in the layer output. Therefore, channels with large activation magnitudes are treated as salient.

### 6.4. Experimental Setup

**Table 4.** Initial AWQ model loading and generation test

| Component                 | Setting                 |
|---------------------------|-------------------------|
| Model                     | Qwen2.5-3B-Instruct-AWQ |
| GPU                       | NVIDIA RTX 5060 Laptop  |
| Available VRAM            | 8151 MiB                |
| Peak Allocated Load       | 2.50 GiB                |
| Peak Reserved Load        | 2.64 GiB                |
| Peak Allocated Generation | 2.69 GiB                |
| Peak Reserved Generation  | 2.84 GiB                |
| Generated Tokens          | 28                      |
| Latency                   | 14.90 s                 |
| Tokens/sec                | 1.88                    |

As an initial validation step, the official AWQ-quantised model `Qwen/Qwen2.5-3B-Instruct-AWQ` was loaded successfully on our local NVIDIA GeForce RTX 5060 Laptop GPU. The environment used PyTorch `2.8.0+cu128`, `CUDA 12.8`, `AutoAWQ 0.2.6`,

and `Transformers 4.42.4`. The model generated a deterministic response using `do_sample=False`, confirming that the official AWQ could be used. Below, we have the characteristics of AWQ shown in Table (4).

### 6.5. Main AWQ Benchmark

The three Qwen2.5-3B-Instruct variants were benchmarked using the same 10 prompts and deterministic decoding settings. For each model, generation used `do_sample = False`, `temperature = None`, and `max_new_tokens = 100`. The models compared were the FP16 baseline, a bitsandbytes NF4 4-bit baseline, and the official AWQ 4-bit quantised model.

**Table 5.** Average inference benchmark results over 10 prompts.

| Model              | Memory (GiB) | Latency (s) | Tokens/s |
|--------------------|--------------|-------------|----------|
| FP16 baseline      | 6.46         | 2.56        | 19.40    |
| NF4 4-bit baseline | 2.53         | 3.29        | 15.42    |
| Official AWQ 4-bit | 2.69         | 29.16       | 1.68     |

Our specific prompts used here can be found in the Appendix B.

#### Interpretation

The results show that both 4-bit methods substantially reduced GPU memory usage compared with the FP16 baseline. The FP16 model used approximately 6.46 GiB of GPU memory, while the NF4 and official AWQ models used approximately 2.53 GiB and 2.69 GiB respectively. This corresponds to about a 60.8% memory reduction for NF4 and a 58.4% memory reduction for AWQ relative to FP16. However, the official AWQ model was much slower in our environment, achieving only 1.68 tokens/s compared with 19.40 tokens/s for FP16 and 15.42 tokens/s for NF4. We suspect that this is due to kernel extensions being unavailable.

### 6.6. Model Variants Perplexity

**Table 6.** Perplexity over 20 blocks of 512 tokens.

| Model              | Avg Loss | Perplexity | Peak Memory (GiB) |
|--------------------|----------|------------|-------------------|
| FP16 baseline      | 2.5159   | 12.38      | 7.30              |
| NF4 4-bit baseline | 2.5943   | 13.39      | 3.36              |
| Official AWQ 4-bit | 2.5961   | 13.41      | 3.36              |

Perplexity was evaluated on the same prompts from Section 6.5 using 20 blocks of up to 512 tokens. The FP16 baseline achieved the lowest perplexity, with a value of 12.38. The NF4 4-bit baseline achieved a perplexity of 13.39, while the official AWQ 4-bit model achieved a perplexity of 13.41. These results show that both 4-bit quantized models preserved our language modelling quality reasonably well, with only a slight increase in perplexity compared with the FP16 baseline. The AWQ and NF4 results were very close in this experiment, suggesting that the official AWQ model did not noticeably outperform NF4 on this small dataset, although both methods substantially reduced GPU memory usage.

### 6.7. Calibration

Calibration is necessary in AWQ because the quantisation process needs examples of real model behaviour before deciding

which weight channels should be protected. When a model is quantised to 4-bit precision, not all weight errors have the same effect on the output. Calibration data is used to run the original full-precision model on representative text samples and measure which internal activation channels are most important.

The purpose of this extension is to investigate whether the choice of calibration samples affects the quality of the activation information available to AWQ. The calibration texts were drawn from the WikiText-2 raw corpus, loaded through the Hugging Face `Salesforce/wikitext` dataset with configuration `wikitext-2-raw-v1`. The calibration pool was constructed from the training split, while perplexity evaluation used the test split. WikiText was introduced by Merity et al. [1] as a language modelling corpus based on high-quality Wikipedia articles.

We should note that we compared 5 different calibration strategies in total. These were:

#### Calibration Strategies

- **Random-small:** a small random calibration set.
- **Random-large:** a larger random calibration set.
- **Long-input:** samples with the largest tokenized sequence lengths.
- **High-activation:** samples that produce the largest activation magnitudes in selected transformer layers.
- **Activation-coverage:** samples selected greedily to cover the largest number of high-activation channels.

#### 6.7.1. Channel Selection

For each monitored layer, We recorded the top 32 hidden channels by mean absolute activation for each calibration sample. Since the model hidden size is 2048, this means only the strongest approximately 1.6% of channels in each monitored layer were treated as high-activation channels.

The top-32 rule was chosen as a practical threshold for all our calibration results in Section 6.7.5. Using all 2048 channels would make almost every sample appear to cover the layer, while using too few channels could make the metric overly sensitive to isolated outliers.

The `Qwen/Qwen2.5-3B-Instruct` model contains 36 transformer layers. For the calibration experiment, we monitored layers 4, 12, 20, and 28 rather than all layers. These layers were chosen to provide a representative spread across the depth of the model, as monitoring every layer would have increased memory usage and runtime on our available laptop GPU.

**NOTE: Again, although explicit, this is shown to help show exactly how these channels are chosen:**

For each calibration sample  $x$ , we monitored four transformer layers:

$$\mathbf{L} = \{4, 12, 20, 28\}.$$

For each monitored layer, we computed the mean absolute activation of every hidden channel and selected the top 32 channels in that layer. Therefore, the top-32 rule was applied separately to each layer.

$$4 \times 32 = 128$$

layer-channel pairs. Let  $C_n$  signify a channel and  $L_n$  signify a layer. Thus:

$$\text{Layer 4: } \{L_4 : C_{318}, \dots\} \quad (17)$$

$$\text{Layer 12: } \{L_{12} : C_{1874}, \dots\} \quad (18)$$

$$\text{Layer 20: } \{L_{20} : C_{1874}, \dots\} \quad (19)$$

$$\text{Layer 28: } \{L_{28} : C_{1874}, \dots\}. \quad (20)$$

#### 6.7.2. Activation - Largest Activation (Referenced Method)

As mentioned in section 6.3, the selected salient channels are chosen via the *largest activations*, this is shown more explicitly below in equation 21. This is the calibration method that J. Lin et al makes use of in their paper [10].

$$S(x) = \frac{1}{|\mathcal{L}|} \sum_{\ell \in \mathcal{L}} \text{mean}(|h_{\ell}(x)|), \quad (21)$$

We take the mean inside each layer as each layer produces many activation numbers, and these are averaged per channel. We then sum this across all layers, and average them across our layers.

#### 6.7.3. Activation - Coverage Scoring Method

For high-activation selection, each candidate calibration sample is passed through the full-precision model, and activation magnitudes are recorded from selected transformer layers. If  $h_{\ell}(x)$  denotes the hidden activation produced by layer  $\ell$  for input sample  $x$ , a simple activation magnitude score is

where  $\mathcal{L}$  is the set of monitored transformer layers. Samples with the largest values of  $S(x)$  are selected for the high-activation calibration set.

For activation-coverage selection, each sample was represented by the set of channels that it strongly activates. The top- $k$  channels by activation magnitude are recorded for each sample. A greedy selection procedure then chooses samples that add the largest number of previously uncovered high-activation channels until the calibration budget is reached.

$$x^* = \arg \max_{x \in \mathcal{C} \setminus \mathcal{S}} \left| \mathcal{A}(x) \setminus \bigcup_{s \in \mathcal{S}} \mathcal{A}(s) \right| \quad (22)$$

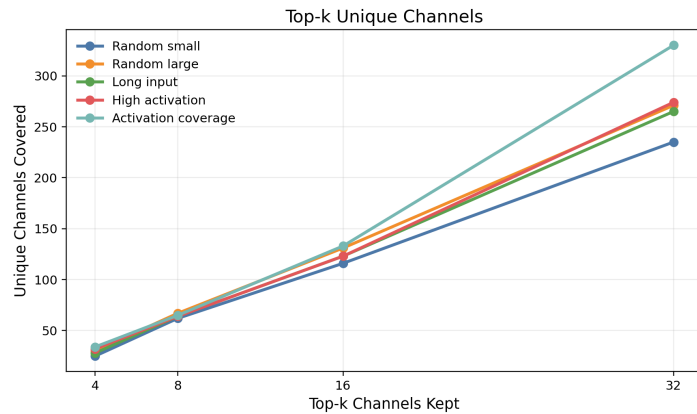


Figure 1. Top-K Channels

We see this in more detail in the below plot in Figure 1, showing that the k-top channels diverge as desired.

Equation 22 gives more compact form for the set of channels that would be formed.

#### 6.7.4. Activation - Long Input

The long-input calibration baseline was included to test whether simply using longer text samples provides better calibration coverage. This strategy selected the 128 candidate samples with the largest tokenised input lengths from the Wikitext calibration pool. The intuition is that longer inputs may contain more varied linguistic patterns and therefore might activate a broader range of model channels. However, the results show that length alone was not a reliable proxy for useful calibration diversity as seen figure 2 below:

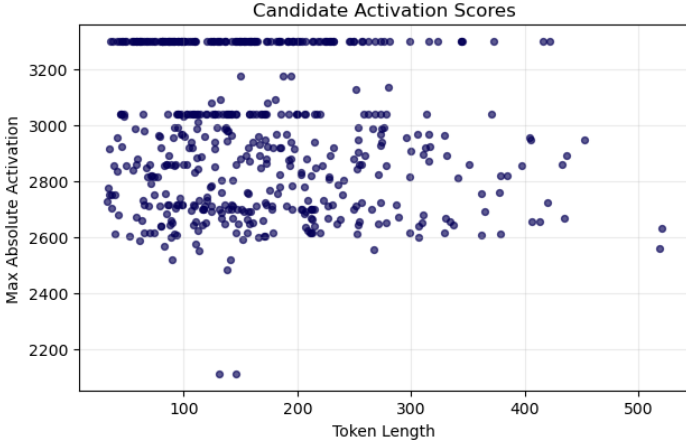


Figure 2. Long Input Activation Scores

This suggests that longer calibration samples do not necessarily expose more diverse internal model behaviour.

#### 6.7.5. Calibration Results

We compared five calibration strategies using 512 Wikitext samples tokenised with the `Qwen/Qwen2.5-3B-Instruct` tokeniser. These were random-small 32, random-large 128, 128 longest inputs, 128 highest-activation samples, and a proposed activation-coverage method. The activation-coverage strategy greedily selected samples that introduced the most previously unseen high-activation layer-channel pairs. As such, there were 8192 possible channels.

The activation-coverage method achieved the highest diversity, covering 330 unique layer-channel pairs. This was higher than the high-activation baseline 274, long-input baseline 265, random-large baseline 271, and random-small baseline 235. This suggests that selecting samples only by length or maximum activation can produce redundant calibration data, while activation coverage provides a broader view of the model’s internal activation behaviour.

We let our custom AWQ method with Activation Coverage be of the following form:

`Custom AWQ = Qwen2.5-3B-Instruct + AutoAWQ + Activation Coverage`

We should note that AutoAWQ is a library that automatically applies the method to a model.

Table 7. Comparison of calibration-selection strategies

| Calibration Strategy | Samples | Activation Channels Covered |
|----------------------|---------|-----------------------------|
| Random Small         | 32      | 235                         |
| Random Large         | 128     | 271                         |
| Long Input           | 128     | 265                         |
| High Activation      | 128     | 274                         |
| Activation Coverage  | 128     | 330                         |

Table 8. Effect of calibration selection on AWQ performance

| Quantisation Strategy   | Perplexity | Peak GPU Memory Used |
|-------------------------|------------|----------------------|
| FP16                    | 12.38      | 7.2952               |
| NF4 4-bit               | 13.39      | 3.3533               |
| Qwen2.5-3B-Instruct-AWQ | 13.41      | 3.3642               |
| Custom AWQ              | 13.22      | 3.3642               |

As such, we can see that the baseline `Qwen2.5-3B-Instruct` model with AutoAWQ and our Activation Coverage sample set correctly yields a lower perplexity, although larger than the FP16 model which is to be expected.

We can thus continue and attempt to build our AWQ model from scratch and see which values for  $\alpha$  in our search as described in section 6.3.1 to further validate this methodology in section 6.8.

### 6.8. Toy Experiment of AWQ

#### 6.8.1. High Activation Calibration

To make the mechanism of AWQ more transparent, we implemented a simplified toy AWQ experiment from scratch on a synthetic linear layer. We isolated the core AWQ idea in a controlled setting. We constructed a linear layer  $y = XW^T$  with 256 input channels and 128 output channels. A small subset of input channels were assigned much larger calibration activations from a normal distribution, representing salient channels whose quantisation error is more likely to affect the layer output. We then compared naive 4-bit groupwise weight quantisation with an activation-aware scaled version inspired by AWQ.

The output error was calculated as previously defined in equation 13.

$$\text{Output Error} = \frac{\|XW^T - X\hat{W}^T\|_2}{\|XW^T\|_2} \quad (23)$$

For the naive baseline, the weights were quantised directly using symmetric groupwise 4-bit quantisation with group size 32. For the toy AWQ method, we first computed an activation importance score for each input channel using the calibration data:

$$a_j = \frac{\mathbb{E}_{x \in \mathcal{D}_{cal}} |x_j|}{\frac{1}{d} \sum_{k=1}^d \mathbb{E}_{x \in \mathcal{D}_{cal}} |x_k|}. \quad (24)$$

A channel scaling factor was then defined as:

$$s_j = s_{X,j}^\alpha \quad (25)$$

Where  $s_X$  was our average across a single channel, and  $\alpha$  controls the strength of the activation-aware scaling as proposed in section

## 6.3.1.

The toy layer used a weight matrix of:

$$W \in \mathbb{R}^{128 \times 256}. \quad (26)$$

Each input channel corresponded to one column of the weight matrix. Therefore, each input channel contained 128 weights. The AWQ-style scaling assigned one scale factor  $s_j$  to each input channel, and this scale was applied to the entire column of 128 weights.

Thus, the toy AWQ scaling used 256 channel scales in total, one for each input channel.

After applying the AWQ-style channel scaling, the weights were quantised using groupwise 4-bit quantisation with group size 32. This is a separate operation from the AWQ channel scaling. As such we had:

$$\frac{256}{32} = 8 \quad (27)$$

groups per output row. Since there were 128 output rows, the total number of quantisation groups was

$$128 \times 8 = 1024 \quad (28)$$

Each quantisation group used its own quantisation scale to map the floating-point weights to signed 4-bit levels.

The exact value of  $\alpha$  that we used for scaling is justified in figure 3.

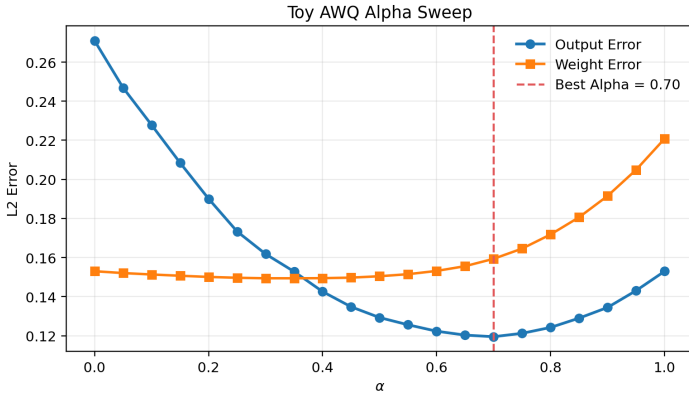


Figure 3. Alpha Sweep

The value of  $\alpha$  was selected by sweeping  $\alpha$  from 0 to 1 in increments of 0.05 and choosing the value that minimised relative output error on a held-out synthetic test set. The best value was  $\alpha = 0.70$ . At this value, high-activation channels received an average scale of 3.5684, while the remaining channels received an average scale of 0.7626. This indicates that the toy AWQ procedure assigned larger scales to channels that were more important under the calibration activation distribution.

The results show that activation-aware scaling substantially reduced output error. Naive 4-bit groupwise quantisation produced a relative output error of 0.2709, while the toy AWQ-scaled 4-bit method reduced this to 0.1195, corresponding to a 55.9% reduction. Interestingly, the relative weight reconstruction error increased slightly from 0.1530 for naive quantisation to 0.1593 for toy AWQ. Minimising raw weight error is not

necessarily the same as preserving the model’s output.

**Table 9.** Toy AWQ experiment comparing naive 4-bit quantisation with activation-aware scaled 4-bit quantisation.

| Method               | Alpha | Weight Error | Output Error |
|----------------------|-------|--------------|--------------|
| Naive Groupwise int4 | 0.00  | 0.1530       | 0.2709       |
| Toy AWQ Scaled int4  | 0.70  | 0.1593       | 0.1195       |

These errors were then cross checked with the error approximated by  $1/s$  in equation 12.

Given  $s = 3.5684$  for this layer, this implies:

$$\frac{\text{AWQ Error}}{\text{RTN Error}} \approx \frac{1}{s} = \frac{1}{3.5684} = 0.2803 \quad (29)$$

$$\frac{\text{AWQ Error}}{\text{RTN Error}} = 0.2163 \quad (30)$$

**Table 10.** Toy AWQ error

| Channel         | Naive Error | AWQ Error | AWQ / Naive Ratio |
|-----------------|-------------|-----------|-------------------|
| High-Activation | 0.3376      | 0.0730    | 0.2163            |
| Other           | 0.2151      | 0.2301    | 1.0765            |
| All             | 0.2223      | 0.2209    | 1.0261            |

The observed mean ratio between AWQ channel error and naive RTN channel error was 0.2163. Thus, the measured error reduction was broadly consistent with the  $1/s$  intuition, although not exact. The discrepancy is expected because the toy implementation uses groupwise quantisation, so scaling one channel can change the quantisation range shared by other weights in the same group.

At the layer-output level, naive RTN quantisation produced a relative output error of 0.2709, while the toy AWQ-scaled version reduced this to 0.1195.

#### Significant Insight

This corresponds to an AWQ/RTN output-error ratio of 0.4411. This ratio is larger than  $1/s$  for the salient channels because the total output error includes all channels, including non-salient channels whose quantisation error slightly increased after scaling.

**Therefore, AWQ does not uniformly reduce all errors. It redistributes quantisation error away from high-activation channels and toward less important channels.**

This error can be reduced with a smaller quantisation block size, but is highly inefficient.

#### 6.8.2. Activation Coverage Calibration and Comparison

Figure 4 compares how different calibration-sample selection strategies affect the toy AWQ scaling procedure. The naive int4 baseline uses no calibration samples and therefore applies no activation-aware scaling. It gives the largest output error, 0.2533. Using calibration samples improves the result because the toy AWQ method can estimate which channels are important and assign larger scales to them before quantisation.

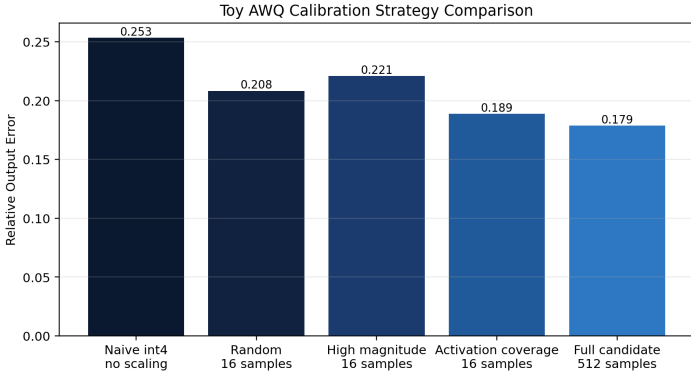


Figure 4. Toy AWQ Error Comparison

Among the 16-sample methods, activation coverage performs best. It covered 115 channels and achieved an output error of 0.1888. This is lower than random selection, which covers 84 channels and gives an error of 0.2083, and lower than high-magnitude selection, which covers only 49 channels and gives an error of 0.2209. This shows that simply choosing the strongest activation samples is not necessarily best, because those samples can be redundant and may repeatedly activate the same channels.

The full candidate pool used all 512 synthetic calibration, and acted as a large reference. It covered all 256 toy input channels and gave the lowest output error, 0.1787. The activation-coverage strategy comes closest to this full-pool result while using only 16 samples, suggesting that coverage-based calibration can approximate the benefit of a much larger calibration set.

strategy initially improves, but its error rises sharply for larger  $\alpha$ . This suggests that high-magnitude calibration overemphasises strongly activated channels. That is, when the scaling strength becomes too large, the model over-protects these redundant channels and hurts the overall quantised output.

#### The Takeaway

The full 512-sample candidate pool gives the best overall curve because it uses far more calibration data and therefore provides the most complete activation estimate. This supports the idea that, under a small sample of calibration, selecting diverse high-activation channel coverage is more useful than simply selecting the largest-magnitude activation samples.

Table 11. Toy AWQ calibration-strategy comparison. Activation coverage gave the lowest output error among the 16-sample calibration methods.

| Calibration Strategy   | Samples | Channels Covered | Best $\alpha$ |
|------------------------|---------|------------------|---------------|
| Naive int4, no scaling | 0       | 0                | 0.00          |
| Random                 | 16      | 84               | 0.75          |
| High magnitude         | 16      | 49               | 0.40          |
| Activation coverage    | 16      | 115              | 0.75          |
| Full candidate pool    | 512     | 256              | 1.00          |

The high-magnitude strategy did not perform as well as activation coverage. Although high-magnitude samples strongly activated the model, they can be redundant, as several selected samples may activate the same small set of channels. In the toy experiment, high-magnitude calibration covered only 49 channels and achieved a test output error of 0.2209. Activation coverage instead selected samples that exposed more distinct high-activation channels, covering 115 channels and reducing test output error to 0.1888. This suggests that calibration diversity can be more useful than simply selecting the samples with the largest activation magnitudes.

#### 6.9. GPTQ vs AWQ Perplexity

We further try to reproduce the comparison between GPTQ and AWQ quantisation as Lin et al show in their work [10] [4]. To test how calibration set size affected quantisation quality, we performed AWQ and GPTQ quantisation runs using random WikiText calibration subsets. The calibration samples were filtered so that each sequence fit within the AutoAWQ calibration window. Both methods used 4-bit weight quantisation with group size 128 and perplexity was evaluated on 20 WikiText-2 test blocks of 512 tokens. As shown in Figure 6 below, increasing the calibration budget from 4 to 32 sequences reduced perplexity for both methods.

AWQ decreased from 13.4230 to 13.3586, while GPTQ decreased from 14.1883 to 13.8933. This suggests that additional calibration data improves the quality of both quantised models, and AWQ achieved lower perplexity than GPTQ.

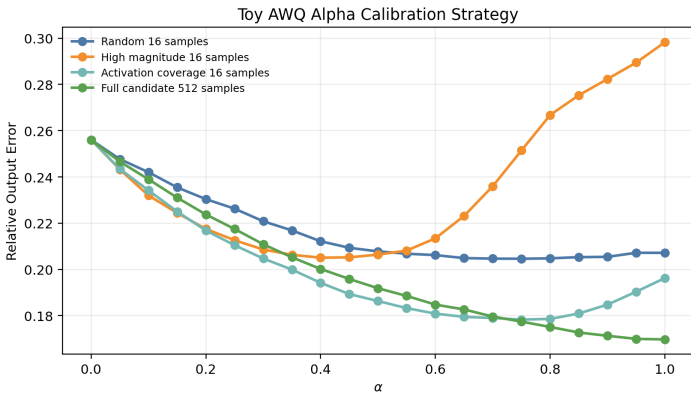


Figure 5. Toy AWQ Error Alpha Comparison

Figure 5 shows how the validation output error changes as the AWQ scaling strength  $\alpha$  is varied for each toy calibration strategy. When  $\alpha = 0$ , no activation-aware scaling was applied, so all strategies begin from the same naive int4 baseline. As  $\alpha$  increases, the scaling factors gave more protection to channels with larger estimated activation importance.

The activation-coverage curve achieved the lowest error among the 16-sample calibration methods across much of the sweep, with its best validation error occurring around  $\alpha = 0.75$ . This indicates that the activation-coverage calibration set provides a better estimate of which channels should be protected. The random 16-sample strategy improves over the naive baseline, but plateaus at a higher error, suggesting that random calibration gives a less complete activation estimate. The high-magnitude

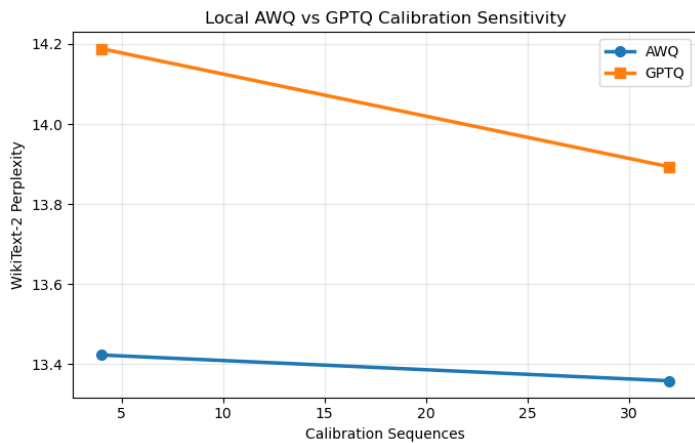


Figure 6. AWQ vs GPTQ Perplexity

### 6.9.1. Qwen Layer-to-Layer Variation

As a final experiment and to validate the performance of AWQ on Qwen, we tested the error reduction across four transformer layers of AWQ, namely 4, 12, 20, and 28. For each layer, we quantised the `self_attn.q_proj` matrix using simulated groupwise 4-bit quantisation with group size 128. The full-precision layer output was used as the reference, and the quantised layer output was compared against it. For each calibration strategy, we swept the AWQ scaling strength  $\alpha$  and recorded the best output error. Table 12 therefore tests whether activation-aware scaling remains useful across different parts of the transformer, rather than only working for the single layer used in the previous experiment.

The improvement was not identical across layers as seen in Table 12. Layers 4 and 12 achieved reductions of roughly 15–16%, while layers 20 and 28 achieved larger reductions of roughly 18–20%. This suggests that later layers were more sensitive to naive 4-bit quantisation in this experiment, giving AWQ-style scaling more room to improve the output.

Table 12. AWQ Performance on different calibration methods for different layers

| Layer | Method              | Channels | Best $\alpha$ | Output Error | Reduction |
|-------|---------------------|----------|---------------|--------------|-----------|
| 4     | Naive RTN           | 0        | 0.00          | 0.0447       | 0.0%      |
| 4     | Random              | 51       | 0.35          | 0.0375       | 16.2%     |
| 4     | High activation     | 60       | 0.30          | 0.0375       | 16.3%     |
| 4     | Activation coverage | 54       | 0.30          | 0.0375       | 16.2%     |
| 12    | Naive RTN           | 0        | 0.00          | 0.0601       | 0.0%      |
| 12    | Random              | 40       | 0.25          | 0.0508       | 15.6%     |
| 12    | High activation     | 53       | 0.30          | 0.0509       | 15.4%     |
| 12    | Activation coverage | 53       | 0.25          | 0.0510       | 15.1%     |
| 20    | Naive RTN           | 0        | 0.00          | 0.0722       | 0.0%      |
| 20    | Random              | 49       | 0.30          | 0.0585       | 18.9%     |
| 20    | High activation     | 51       | 0.30          | 0.0590       | 18.3%     |
| 20    | Activation coverage | 53       | 0.30          | 0.0589       | 18.4%     |
| 28    | Naive RTN           | 0        | 0.00          | 0.0705       | 0.0%      |
| 28    | Random              | 57       | 0.30          | 0.0564       | 20.0%     |
| 28    | High activation     | 62       | 0.30          | 0.0564       | 20.1%     |
| 28    | Activation coverage | 61       | 0.30          | 0.0564       | 20.0%     |

### Interpretation

The differences between random, high-activation, and activation-coverage calibration were small in this single-layer

test. This implies that, once a small calibration set is enough to estimate the most important channels for one matrix, the main benefit comes from the AWQ scaling mechanism itself. Activation coverage remained competitive, but it did not dominate as clearly as in the toy experiment, where calibration diversity had a stronger effect.

## 7. LIMITATIONS AND FUTURE WORK

This project has several limitations. Most experiments were run on a laptop GPU with approximately 8 GB of VRAM, which constrained the calibration sequence length, batch size, and number of evaluation prompts. The custom AWQ run was therefore configured conservatively, using 128 activation-coverage calibration samples and processing calibration data in a memory-efficient way. The activation-coverage method was evaluated mainly using WikiText-2 perplexity and a small deterministic generation test, so the results may not fully capture task performance or long-context behaviour. Covering more high-activation channels suggests a broader calibration signal, but it does not prove that every additional channel is causally important.

Future work could extend this study by testing activation-coverage calibration on more datasets, larger calibration pools, tasks such as summarisation, reasoning, and classification. It would also be useful to repeat the custom AWQ quantisation across several random seeds to check whether the observed perplexity improvement is robust. Another direction would be to integrate the activation-coverage selection method more directly into the AutoAWQ pipeline, allowing the calibration set to be selected automatically before quantisation.

## 8. CONCLUSION

This project investigated AWQ as a practical method for reducing LLM memory usage through activation-aware 4-bit weight quantisation. The baseline experiments showed that quantised models substantially reduced GPU memory compared with the FP16 baseline, making inference more feasible on limited hardware. Using this activation-coverage set with AutoAWQ quantisation run produced a custom AWQ model with lower WikiText-2 perplexity than the official AWQ model, while maintaining similar peak GPU memory.

The toy AWQ experiments further explained why activation-aware scaling works. Although AWQ did not minimise global weight error, it reduced output error by protecting high-activation channels whose quantisation errors have the largest effect on the layer output. The GPTQ vs AWQ and layer to layer experiments further validated the performance of AWQ under different calibration conditions. Overall, the results support the main conclusion that AWQ performance depends not only on the quantisation algorithm, but also on the representativeness and diversity of the calibration data used to estimate activation importance.

## 9. CODE AVAILABILITY

The Jupyter notebook used for model implementation, experimentation, and result generation is available in the project repository at:

[https://github.com/JamesCondos/SOML\\_Final\\_Project\\_Paper](https://github.com/JamesCondos/SOML_Final_Project_Paper)

## REFERENCES

- [1] S. Merity, C. Xiong, J. Bradbury, and R. Socher, *Pointer sentinel mixture models*, 2016. arXiv: 1609.07843 [cs.CL].
- [2] T. Wolf *et al.*, “Transformers: State-of-the-art natural language processing”, in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2020, pp. 38–45.
- [3] E. J. Hu *et al.*, “LoRA: Low-rank adaptation of large language models”, *arXiv preprint arXiv:2106.09685*, 2021. arXiv: 2106.09685 [cs.CL].
- [4] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, “Gptq: Accurate post-training quantization for generative pre-trained transformers”, *arXiv preprint arXiv:2210.17323*, 2022.
- [5] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, “Smoothquant: Accurate and efficient post-training quantization for large language models”, *arXiv preprint arXiv:2211.10438*, 2022.
- [6] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “Qlora: Efficient finetuning of quantized llms”, *arXiv preprint arXiv:2305.14314*, 2023.
- [7] C. Hansen, *Autoawq*, <https://github.com/casper-hansen/AutoAWQ>, 2023.
- [8] A. Q. Jiang *et al.*, “Mistral 7b”, *arXiv preprint arXiv:2310.06825*, 2023. [Online]. Available: <https://arxiv.org/abs/2310.06825>.
- [9] Mistral AI, *Mistral-7b-instruct-v0.1*, <https://huggingface.co/mistralai/Mistral-7B-Instruct-v0.1>, Hugging Face model card. Accessed: 2026-05-08, 2023.
- [10] J. Lin *et al.*, “AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration”, in *Proceedings of the 7th Conference on Machine Learning and Systems*, Best Paper Award, Santa Clara, CA, USA, 2024.
- [11] K. Sun and M. Dredze, “Amuro & char: Analyzing the relationship between pre-training and fine-tuning of large language models”, *arXiv preprint arXiv:2408.06663*, 2024. arXiv: 2408.06663 [cs.CL]. [Online]. Available: <https://arxiv.org/abs/2408.06663>.
- [12] Hugging Face, *Supervised fine-tuning*, <https://huggingface.co/learn/llm-course/en/chapter11/1>, Accessed: 2026-05-21, 2026.

## A. LLM GENERATION OUTPUTS

This appendix reports the raw text-generation outputs used to compare the effect of decoding parameters on model behaviour. The same prompt was used for each generation, while temperature, maximum token count, and sampling configuration were varied. These outputs support the discussion in Section 3.1.

**Table 13.** Generation settings used for the LLM response comparison.

| Output | Temperature | Max tokens | Sampling |
|--------|-------------|------------|----------|
| A.1    | 0.7         | 1000       | Enabled  |
| A.2    | 0.1         | 900        | Enabled  |
| A.3    | 1.0         | 1000       | Enabled  |
| A.4    | –           | 1000       | Disabled |

### A.1. Baseline Sampling Output

Generation command:

```
print(f"Response:\n{generate_text(my_prompt, max_tokens=1000, temperature=0.7)}")
```

Introduction to Large Language Models (LLMs) for Students with Maths and Programming Background

Hello Students,

If you have a solid foundation in mathematics and programming, you're well-prepared to dive into the world of Large Language Models (LLMs). LLMs are at the forefront of natural language processing (NLP) and artificial intelligence (AI), and they're poised to transform how computers understand and generate human language.

Here's a structured introduction to LLMs tailored for you:

- Background Knowledge**
  - Understand the basics of NLP: Familiarize yourself with techniques like tokenization, part-of-speech tagging, and syntactic parsing.
  - Recognize the importance of machine learning: Have a grasp on supervised and unsupervised learning, neural networks, and gradient descent.
  - Comfort with programming: Know at least one programming language (Python is common in the NLP field) and understand libraries like TensorFlow or PyTorch.
- What are Large Language Models?**
  - LLMs are AI models designed to interpret, generate, and respond to human language.
  - They are trained on vast amounts of text data, learning patterns and relationships between words.
  - The model's size is relative; it refers to the number of parameters (weights) in the neural network, which can range from millions to billions.
- How do LLMs Work?**
  - Neural Networks: Deep neural networks with multiple layers are used, which learn hierarchical representations of language.
  - Transformer Architecture: Many LLMs, like GPT (Generative Pre-trained Transformer) and BERT (Bidirectional Encoder Representations from Transformers), rely on the transformer architecture.
  - Attention Mechanisms: The attention mechanism allows the model to focus on different parts of a sentence when generating responses or translations, similar to how humans pay attention to different words when understanding a sentence.
- Training LLMs**
  - Pre-training: LLMs are typically pre-trained on vast corpora of text data. This phase involves learning the structure of language without specific task labels.
  - Fine-tuning: After pre-training, models can be fine-tuned on a smaller, task-specific dataset to perform particular tasks like translation, summarization, or question answering.
  - Reinforcement Learning: Sometimes, LLMs are further refined using reinforcement learning to improve their performance based on feedback.
- Applications of LLMs**
  - Virtual Assistants: Chatbots and virtual helpers that understand and respond to natural language queries.
  - Content Generation: Creating articles, stories, or code comments.
  - Language Translation: Translating text between languages in real-time.
  - Sentiment Analysis: Analyzing the tone and emotions in text data.
  - Accessibility: Assisting individuals with disabilities by translating text to speech or speech to text.
- Challenges and Considerations**
  - Data Bias: LLMs may inherit biases present in their training data.
  - Ethical Implications: Consider the potential misuse of language models for generating false information.
  - Computational Resources: Training LLMs requires substantial computational power.

7. **Practical Work**
  - Experiment: Try using an open-source LLM like Hugging Face's transformers library. Implement a simple NLP task to see how these models work.
  - Research: Keep up-to-date on the latest developments and papers in the field of NLP and LLMs.
8. **Coursework and Further Learning**
  - Enroll in courses: Many institutions and online platforms offer specialized courses in NLP and LLMs.
  - Practicals: Engage in coding challenges and Kaggle competitions to apply what you've learned.

Remember, the field of LLMs is rapidly evolving. As you learn, stay curious and adaptable. Keep experimenting, and you'll contribute to the exciting future of machine understanding of language.

**Code 2.** Generated response using baseline sampling with temperature 0.7 and max\_tokens 1000.

## A.2. Low-Temperature Sampling Output

Generation command:

```
print(f"Response:\n{generate_text(my_prompt, max_tokens=900, temperature=0.1)}")
```

Introduction to Large Language Models (LLMs) for Students with Maths and Programming Background

Hello students,

Welcome to this introduction to Large Language Models (LLMs), a fascinating intersection of natural language processing (NLP), machine learning (ML), and programming. If you have a background in mathematics and programming, you're well-positioned to understand and appreciate the complexities and potential of LLMs.

Here's a structured outline to help you navigate through this topic:

1. **Understanding Language Models**
  - **Definition**: A language model is a statistical model that predicts the likelihood of a sequence of words. It's a foundational concept in NLP.
  - **Mathematical Basis**: Language models often use probability distributions, such as n-gram models, which calculate the probability of a word given the previous n-1 words.
  - **Connection to Programming**: Implementing language models involves writing code that can process text, tokenize it, and apply algorithms to predict word sequences.
2. **From NLP to LLMs**
  - **Evolution**: Discuss how traditional NLP models have evolved into LLMs, which are much larger and more complex.
  - **Scaling Up**: Explore how LLMs are trained on vast datasets using distributed computing and advanced optimization techniques.
3. **Machine Learning and LLMs**
  - **Supervised Learning**: Understand how LLMs are trained using large labeled datasets.
  - **Unsupervised Learning**: Learn about self-supervised learning techniques like masked language modeling (MLM) used in models like BERT.
  - **Reinforcement Learning**: Discuss how LLMs can be fine-tuned using reinforcement learning for specific tasks.
4. **Architecture of LLMs**
  - **Transformers**: Introduce the transformer architecture, which is the backbone of many LLMs.
  - **Attention Mechanism**: Explain how the attention mechanism allows the model to focus on different parts of the input sequence.
  - **Scalability**: Discuss how LLMs scale with more layers, attention heads, and larger vocabularies.
5. **Training LLMs**
  - **Data Preprocessing**: Cover the importance of cleaning and tokenizing data.
  - **Optimization Algorithms**: Discuss the use of algorithms like Adam, SGD, and learning rate schedules.
  - **Regularization Techniques**: Understand why techniques like dropout and weight decay are used to prevent overfitting.
6. **Evaluation Metrics**
  - **Perplexity**: Explain how perplexity is used to measure the performance of language models.
  - **Benchmarks**: Discuss common benchmarks like GLUE, SuperGLUE, and SQuAD for evaluating LLMs.
7. **Applications of LLMs**
  - **Natural Language Understanding (NLU)**: Explore how LLMs are used for tasks like sentiment analysis, question answering, and language translation.
  - **Natural Language Generation (NLG)**: Discuss applications like text summarization, chatbots, and creative writing.
  - **Transfer Learning**: Understand how LLMs can be fine-tuned for specific tasks with relatively little data.

```

8. Ethical Considerations and Limitations
- Bias and Fairness: Discuss the potential for bias in LLMs and the importance of fairness in AI.
- Interpretability: Explore the challenges of understanding and explaining the decisions made by LLMs.
- Environmental Impact: Consider the computational resources required to train and run LLMs.

9. Practical Workshop
- Hands-on Experience: Engage in coding exercises using libraries like Hugging Face's Transformers, TensorFlow, or PyTorch.
- Project Ideas: Suggest projects that allow you to apply your knowledge of LLMs, such as building a chatbot or summarizing articles.

10. Conclusion
- Future Directions: Discuss ongoing research and future trends in LLMs.
-

```

**Code 3.** Generated response using low-temperature sampling with temperature 0.1 and max\_tokens 900.

### A.3. High-Temperature Sampling Output

Generation command:

```
print(f"Response:\n{generate_text(my_prompt, max_tokens=1000, temperature=1)}")
```

Introduction to Large Language Models (LLMs) for Students with Background in Maths and Programming

Hey there, students! You have a solid foundation in maths and programmingso, lets dive into the exciting world of Large Language Models (LLMs). This is an interdisciplinary area that combines your math skills, algorithmic understanding, and coding proficiency with natural language processing (NLP). Heres how you can understand and explore LLMs:

### What is a Large Language Model (LLM)?

At its core, an LLM is a statistical model designed to understand, process, and generate human-like text. It achieves this by learning from vast amounts of text data. Think of it as teaching the computer to comprehend human language pattern and generate relevant responses.

### The Building Blocks: Neural Networks

Your mathematical knowledge comes in handy as LLMs are grounded on neural networks, specifically, deep learning architectures like Recurrent Neural Networks (RNNs), Convolutional Neural Networks (CNNs), and Transformers.

- **Neural Networks**: Understanding backpropagation, activation functions (like ReLU, Sigmoid, Softmax), and gradients is crucial. These concepts help the LLM learn and adjust weights during training.
- **Transfer Learning**: LLMs typically utilize pre-trained models, applying transfer learning to fine-tune on specific tasks, which saves time and computational resources.

### Programming for LLMs

Programming skills are needed to:

- **Select and Fine-tune Pretrained Models**: Use frameworks like TensorFlow or PyTorch to load models and adapt them to your needs.
- **Data Management**: Utilize libraries like Pandas and NumPy for data processing and handling.
- **Implement the Model**: Build the forward-pass and understand how the softmax layer translates logits to probabilities across categories (like next-word prediction).

### Mathematical Fundamentals for LLMs

Here's where you plug in your math:

- **Optimization**: Knowledge of gradient descent and its variants (SGD, Adam, RMSprop) to train the neural network.
- **Regularization Techniques**: Like L1 and L2, dropout, and Batch Normalization to prevent overfitting.
- **Probabilistic Foundations**: Understanding of likelihood functions and Bayesian inference helps in improving prediction accuracy.

### Training a LLM

Here's a simplified overview of the process:

1. **Dataset Acquisition**: Large text corpora from sources like Wikipedia, books, and online articles.
2. **Data Preprocessing**: Cleaning, tokenization, and encoding your text data.
3. **Building the LLM**: Adopting a Transformer-based architecture (like GPT).
4. **Optimization and Training**: Use your algorithmic knowhow to train with objectives like Maximum Likelihood Estimation (MLE) or Next Sentence Prediction.
5. **Evaluation**: Metrics such as BLEU, ROUGE for language models evaluate performance against human-generated text.

```

Tools and Resources
Get started with these Python libraries:
- **NLTK**, **Spacy**, **Gensim**: Great for NLP tasks.
- **Transformers Library**: Prebuilt models and tokenizers for deep learning-powered NLP.
- **Datasets and Text Generation APIs**: Like Hugging Face's Transformers, which has a wide range of pre-trained models.

Interactive Projects: Engage in projects like chatbots, automated summarization, translation services, or creating custom
↳ language applications.

Remember, language models are powerful, but also require ethical considerations, like avoiding biases and privacy violations.

There you go! With your mathematical skills and programming expertise, you can delve into the ever-expanding universe of LLMs.
↳ Happy coding!

Feel free to explore, experiment, and keep building your knowledge on these topics. Who knows, you might even create your next
↳ groundbreaking NLP model!

```

**Code 4.** Generated response using high-temperature sampling with temperature 1.0 and max\_tokens 1000.

## A.4. Deterministic Decoding Output

### Generation command:

```
print(f"Response:\n{generate_text(my_prompt, max_tokens=1000, do_sample_state=False)}")
```

Introduction to Large Language Models (LLMs) for Students with Maths and Programming Background

Hello students,

Welcome to this introduction to Large Language Models (LLMs), a fascinating intersection of natural language processing (NLP),  
↳ machine learning (ML), and computer science. If you have a background in mathematics and programming, you're well-positioned  
↳ to dive into the world of LLMs. Here's a structured outline to help you understand the key concepts and technologies  
↳ involved:

1. **Overview of Machine Learning and NLP**
  - **Machine Learning Basics:**
    - Supervised vs. Unsupervised Learning
    - Reinforcement Learning
    - Feature Engineering
    - Model Evaluation Metrics (e.g., accuracy, precision, recall)
  - **Natural Language Processing (NLP):**
    - Text Preprocessing (tokenization, stemming, lemmatization)
    - Feature Extraction (bag-of-words, TF-IDF)
    - Basic NLP Models (Naive Bayes, SVM)
2. **Introduction to Deep Learning**
  - **Neural Networks:**
    - Architecture (input, hidden, output layers)
    - Activation Functions (ReLU, sigmoid, softmax)
    - Backpropagation and Gradient Descent
  - **Convolutional Neural Networks (CNNs):**
    - Applications in image processing
  - **Recurrent Neural Networks (RNNs):**
    - Sequence modeling (e.g., language modeling)
  - **Long Short-Term Memory (LSTM) and Gated Recurrent Units (GRUs):**
    - Handling long-term dependencies
3. **Understanding Transformers and Attention Mechanisms**
  - **Transformer Architecture:**
    - Self-attention and its role in parallelizing sequence processing
    - Positional Encoding
    - Multi-Head Attention
  - **BERT, GPT, and other Transformer-based Models:**
    - Pre-training and fine-tuning
    - Contextual embeddings
    - Transfer Learning
4. **Training LLMs**

```

- **Dataset Preparation:**
 - Text Corpus Collection
 - Annotation and Labeling
- **Model Architecture:**
 - Designing LLM architectures
 - Hyperparameter Tuning
- **Training Procedures:**
 - Batch Processing
 - Learning Rate Scheduling
 - Regularization Techniques (dropout, weight decay)
- **Evaluation and Interpretation:**
 - Perplexity and other metrics
 - Qualitative analysis (e.g., BLEU score for translation)

5. **Applications of LLMs**
- **Natural Language Generation (NLG):**
 - Text summarization
 - Creative writing
- **Machine Translation:**
 - Cross-lingual understanding
- **Question Answering Systems:**
 - Chatbots and virtual assistants
- **Sentiment Analysis:**
 - Social media monitoring
- **Information Retrieval:**
 - Search engines

6. **Ethical Considerations and Future Directions**
- **Bias and Fairness:**
 - Data representation
 - Mitigation strategies
- **Explainability and Transparency:**
 - Model interpretability
 - Ethical AI
- **Advancements in LLMs:**
 - Continual learning
 - Multimodal models
 - Interactive and context-aware systems

7. **Hands-on Projects and Resources**
- **Project Ideas:**
 - Building a chatbot
 - Creating a summarization tool
- **Tools and Libraries:**
 - TensorFlow, PyTorch
 - Hugging Face Transformers
 - SpaCy, NLTK
- **Datasets:**
 - Common Crawl
 - Microsoft Common Crawl
 - Stanford Natural Language Inference (SNLI)

By understanding these concepts and engaging with the provided resources, you'll be well on your way to exploring the exciting
↳ world

```

**Code 5.** Generated response using deterministic decoding with sampling disabled and max\_tokens 1000.

## B. PROMPT SELECTION FOR AWQ COMPARISON

**Table 14.** Evaluation Prompts

| No. | Prompt                                                                                                                        |
|-----|-------------------------------------------------------------------------------------------------------------------------------|
| 1   | Explain to a uni student what a heat pump does in three sentences.                                                            |
| 2   | Solve this: A car leaves at 4 pm and drives at 80 km/h for 1.5 hours. How far does it go?                                     |
| 3   | Summarise this sentence: The bridge design was changed because the first version bent too much under load.                    |
| 4   | Classify the sentiment as Positive or Negative: The lab took ages to set up, but the data we got was actually pretty good.    |
| 5   | Explain why checking your measurements twice can give a better engineering result.                                            |
| 6   | Write one sentence describing the trade-off between cost and strength when choosing a building material.                      |
| 7   | Answer with only A, B, C, or D: Which option is used to measure temperature? A. Ruler B. Thermometer C. Stopwatch D. Compass. |
| 8   | Give one limitation of testing a design using only one trial.                                                                 |
| 9   | Explain what battery capacity means in a phone or laptop.                                                                     |
| 10  | State one reason a smaller motor might not always use less energy in a real system.                                           |